

1. TECNOLOGIAS UTILIZADAS

1.1. ORM (OBJECT-RELATIONAL MAPPING)

O ORM (Object-Relational Mapping) é uma técnica de programação que permite converter dados entre sistemas orientados a objetos e bancos de dados relacionais. Ele funciona como uma camada intermediária que traduz objetos do código da aplicação em registros de tabelas do banco de dados, e vice-versa. Isso permite que os desenvolvedores trabalhem com objetos da linguagem de programação em vez de escrever diretamente comandos SQL (FOWLER, 2006).

Com o uso do ORM, cada tabela do banco de dados geralmente corresponde a uma classe no sistema, enquanto cada linha da tabela representa um objeto dessa classe. As colunas da tabela são mapeadas para atributos do objeto. Dessa forma, operações como inserir, atualizar, consultar ou excluir dados podem ser feitas por meio de métodos e objetos da linguagem de programação (DJANGO SOFTWARE FOUNDATION, 2026).

No desenvolvimento do sistema de reservas do carrinho de sorvete, será utilizado o conceito de ORM para realizar a integração entre a aplicação e o banco de dados. Esse padrão permite que as informações do sistema sejam representadas por meio de objetos no código, enquanto os dados são armazenados em tabelas no banco de dados relacional. Dessa forma, cada entidade importante do sistema, como reservas, sabores e carrinhos disponíveis, poderá ser representada por classes dentro da aplicação.

Para o sistema proposto, o ORM será utilizado para estruturar o relacionamento entre as principais entidades do projeto, como reservas, clientes, carrinhos e sabores, garantindo que as informações possam ser armazenadas, consultadas e atualizadas de maneira organizada no banco de dados. A definição detalhada da tecnologia e da implementação prática será realizada nas etapas posteriores do desenvolvimento.

1.2.MVC (MODEL-VIEW-CONTROLLER)

O MVC (Model–View–Controller) é um padrão de arquitetura de software utilizado para organizar e estruturar aplicações, separando responsabilidades em três componentes principais: Model, View e Controller. Essa separação facilita o desenvolvimento, manutenção e escalabilidade do sistema, pois cada parte possui uma função específica dentro da aplicação. O objetivo do MVC é reduzir o acoplamento entre a interface do usuário e a lógica de negócio (REENSKAUG, 1979).

O Model é responsável por representar os dados da aplicação e as regras de

negócio. Ele gerencia a lógica relacionada aos dados, como validações, cálculos e comunicação com o banco de dados. Em outras palavras, o Model é a parte que define como os dados são armazenados, manipulados e processados, independentemente de como eles serão exibidos ao usuário.

A View é a camada responsável pela interface com o usuário, ou seja, pela apresentação dos dados. Ela exibe as informações vindas do Model de forma visual, como páginas web, telas ou relatórios. Já o Controller atua como intermediário entre o Model e a View, recebendo as ações do usuário, processando essas solicitações e atualizando o Model ou a View conforme necessário.

No sistema da sorveteria desenvolvido para o projeto, o Model representa as entidades do sistema, como produtos (sorvetes, sabores e adicionais), clientes, pedidos e vendas. Essa camada é responsável por armazenar e manipular as informações no banco de dados, garantindo que os dados cadastrados estejam corretos e disponíveis para o restante da aplicação.

A View corresponde às telas que o usuário utiliza para interagir com o sistema, como as interfaces de cadastro de produtos, registro de pedidos e visualização das vendas. No caso do sistema da sorveteria, essas telas permitem que o atendente selecione os sabores, registre pedidos dos clientes e visualize informações de forma clara e organizada.

Já o Controller atua como intermediário entre o Model e a View. Quando o usuário realiza alguma ação, como cadastrar um novo sabor de sorvete ou registrar uma venda, o Controller recebe essa solicitação, processa as informações e aciona o Model para salvar ou buscar os dados necessários. Depois disso, o Controller atualiza a View com as informações corretas. Dessa forma, o sistema da sorveteria mantém uma estrutura organizada, facilitando futuras melhorias, manutenção do código e inclusão de novas funcionalidades.

1.3.DJANGO

1.3.1. MVR DO DJANGO

A arquitetura MVT (Model-View-Template) é o padrão utilizado pelo framework Django para organizar o desenvolvimento de aplicações web. Esse modelo tem como objetivo separar as responsabilidades do sistema em diferentes camadas, facilitando a manutenção, organização e escalabilidade do projeto. Cada componente possui uma função específica, o que contribui para um código mais limpo e estruturado (DJANGO SOFTWARE FOUNDATION, 2026).

O componente Model é responsável pela definição da estrutura dos dados e pela lógica de negócio relacionada ao banco de dados. Nele são declaradas as classes que representam as entidades do sistema, como clientes, reservas e produtos. Além disso, o Model utiliza o ORM do Django para realizar operações no banco de dados de forma automatizada, sem a necessidade de escrever comandos SQL diretamente.

O componente View atua como intermediário entre o Model e o Template. Ele é responsável por processar as requisições do usuário, acessar os dados necessários no Model e retornar uma resposta adequada. As Views podem conter regras de negócio, validações e lógica de controle, sendo fundamentais para o funcionamento da aplicação.

O Template é a camada responsável pela interface visual do sistema. Ele define como os dados serão apresentados ao usuário, utilizando HTML combinado com a linguagem de template do Django. Essa separação permite que a parte visual seja alterada sem interferir na lógica do sistema, garantindo maior flexibilidade no desenvolvimento.

A interação entre esses três componentes ocorre de forma integrada: o usuário faz uma requisição, a View processa essa requisição, consulta o Model quando necessário e retorna os dados para o Template, que os exibe na interface. Dessa forma, o padrão MVT proporciona uma estrutura organizada, eficiente e adequada para o desenvolvimento de aplicações web robustas.

1.3.2. ORM DO DJANGO

O ORM (Object-Relational Mapping) do Django é uma ferramenta que permite a interação com o banco de dados utilizando programação orientada a objetos. Em vez de escrever comandos SQL diretamente, o desenvolvedor manipula classes e objetos em Python, tornando o desenvolvimento mais simples, intuitivo e produtivo (DJANGO SOFTWARE FOUNDATION, 2026).

No Django, o ORM é implementado por meio das classes definidas no Model. Cada classe representa uma tabela no banco de dados, enquanto os atributos da classe correspondem às colunas dessa tabela. Dessa forma, ao criar um objeto em Python, o desenvolvedor está, na prática, criando um registro no banco de dados.

Uma das principais vantagens do ORM é a abstração do banco de dados. O desenvolvedor não precisa se preocupar com diferenças entre sistemas como MySQL, PostgreSQL ou SQLite, pois o Django se encarrega de traduzir as operações realizadas em código Python para comandos SQL compatíveis com o banco utilizado.

Além disso, o ORM facilita operações comuns como criação, leitura, atualização

e exclusão de dados (CRUD). Por meio de métodos simples, é possível realizar consultas, aplicar filtros e manipular dados com poucas linhas de código. Isso reduz a complexidade e diminui a chance de erros, especialmente para desenvolvedores iniciantes.

Em resumo, o ORM do Django simplifica significativamente o acesso ao banco de dados, tornando o código mais legível e organizado. Ele permite que o desenvolvedor foque na lógica da aplicação, enquanto o framework gerencia a comunicação com o banco de dados, garantindo eficiência e produtividade no desenvolvimento.

1.4. LGPD

A proteção dos dados pessoais dos usuários é um aspecto fundamental no desenvolvimento do sistema, especialmente considerando as diretrizes estabelecidas pela Lei Geral de Proteção de Dados Pessoais (LGPD – Lei nº 13.709/2018). O sistema foi projetado com o objetivo de garantir a segurança, privacidade e integridade das informações coletadas, adotando boas práticas de segurança da informação e tratamento de dados (BRASIL, 2018).

Os dados coletados pelo sistema, como nome, telefone, endereço e e-mail, são utilizados exclusivamente para a realização e gerenciamento das reservas solicitadas pelos usuários. Não há compartilhamento dessas informações com terceiros sem o devido consentimento do titular, assegurando que o tratamento dos dados esteja alinhado com os princípios da finalidade e necessidade previstos na LGPD.

Para garantir a segurança das informações, o sistema adota mecanismos de proteção como validação de dados de entrada, controle de acesso por níveis de usuário (administrador e cliente) e restrição de acesso às informações sensíveis. Além disso, recomenda-se a utilização de conexões seguras (HTTPS) e armazenamento protegido no banco de dados, reduzindo riscos de acesso não autorizado.

O acesso aos dados é limitado apenas a usuários autorizados, sendo o administrador responsável pelo gerenciamento das informações e das reservas. Também são implementadas medidas para evitar acessos indevidos, como autenticação de usuários e proteção das rotas do sistema, garantindo que apenas pessoas autorizadas possam visualizar ou modificar os dados.

Por fim, o sistema prevê a transparência no uso das informações, permitindo que o usuário tenha ciência sobre quais dados estão sendo coletados e sua finalidade. Dessa forma, o projeto busca estar em conformidade com a LGPD, promovendo a segurança da informação e o respeito à privacidade dos usuários.

REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. *Lei Geral de Proteção de Dados Pessoais (LGPD)*. Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018.

[Lei Geral de Proteção de Dados – Planalto](#)

DJANGO SOFTWARE FOUNDATION. *Django documentation*. Disponível em: <https://docs.djangoproject.com/>. Acesso em: 18 maio 2026.

[Documentação Oficial do Django](#)

FOWLER, Martin. *Padrões de arquitetura de aplicações corporativas*. Porto Alegre: Bookman, 2006.

REENSKAUG, Trygve. *Models – Views – Controllers*. Xerox PARC, 1979.